

Assignment 1 (grupal part)

Instructions in English provided below

50% out of 100% of lab part (P0 + P1).

Enunciado (Español)

Introducción

En esta práctica aprenderemos sobre cobertura de código para la realización de pruebas y control de versiones de nuestra aplicación. Comenzaremos analizando la cobertura de código (*code coverage*) en un entorno heredado utilizando C y herramientas de análisis web, para posteriormente realizar una migración a un ecosistema moderno con Python y Visual Studio Code. En este entorno, implementaremos pruebas automatizadas para visualizar los posibles problemas que pueda presentar el código en tiempo real. Finalmente, cerraremos el ciclo de vida gestionando la evolución del software mediante Git y GitHub, simulando un flujo de trabajo colaborativo que asegure la trazabilidad y la integridad de cada cambio introducido.

Tareas a desarrollar

Parte 1: Cobertura de pruebas en entorno heredado

Se aporta el código de un programa que acepta las longitudes enteras de los tres lados de un triángulo y lo clasifica como equilátero, isósceles o escaleno (o indica que no se forma un triángulo). Se probará el código utilizando la herramienta web de medición de cobertura de pruebas [ExpCov](#) (analizador de cobertura de ramas), empleando los siguientes valores de entrada:

- Caso 1: 6, 5, 10
- Caso 3: 3, 3, 4
- Caso 2: 6, 6, 6
- Caso 4: 4, 3, 0
- Caso 5: 8, 2, 4

Si se detectara algún defecto durante la ejecución de pruebas en las tareas siguientes, debe documentarse haciendo referencia a la especificación del caso pertinente donde se observe que la salida esperada y la salida obtenida no coinciden, y sugiriendo la causa del problema.

Tareas de prueba del código:

1) Verificar y documentar la cobertura detallada alcanzada con este conjunto de casos, mostrando los informes e indicadores de la herramienta, así como señalando exactamente qué opciones del código no fueron ejecutadas. Los casos deben especificarse indicando los datos de entrada, la salida esperada y la salida obtenida.

2) Se añadirá el número mínimo de casos de entrada necesarios para intentar alcanzar el 100% de cobertura de decisiones y condiciones. Se especificarán los casos adicionales, justificando su elección en base a un diseño razonado. Además, al igual que en la fase anterior, se

documentará la cobertura alcanzada y los indicadores del conjunto total de casos (los anteriores más los nuevos añadidos) con detalles de la herramienta para demostrar que se ha logrado la cobertura completa.

3) Se realizará una última fase de prueba donde deben añadirse casos de prueba adicionales a los mínimos anteriores para la cobertura total y que puedan tener gran probabilidad de detectar problemas en el código, **únicamente** usando lógica de caja negra de **valores límite** y de **casos especiales**. Deberá documentarse la lógica para proponer los casos y su documentación de ejecución (entrada usada, salida correcta esperada y salida real obtenida) sin informar de cobertura de caja blanca con ExpCov.

Código:

```
#include <math.h>
#include <stdio.h>

int
checktriangle (int a, int b, int c)
{
    int A, B, C;
    if ((c < a + b) && (b < a + c) && (a < c + b))
    {
        if (a == b && a == c)
            {printf ("Triangulo equilatero \n");}
        else if (a == b || b == c)
            {printf ("Triangulo isosceles \n");}
        else
            {printf ("Triangulo escaleno \n");}
    }
    else
        {printf ("No es un triangulo \n");}
    return 0;
}

int
main ()
{
    int a, b, c, i, n;
    printf ("Numero de casos de prueba: \n");
    scanf ("%d", &n);
    printf ("%d", n);
    printf ("\n");

    for (i = 1; i < n + 1; i++)
        {printf ("Marcar los valores de longitud de los lados del triangulo (uno por linea): \n");
        scanf ("%d", &a); printf ("%d", a);
        scanf ("%d", &b); printf ("%d", b);
        scanf ("%d", &c); printf ("%d: \n", c);
        checktriangle (a, b, c);};
    return 0;
}
```

Parte 2: Migración y cobertura de pruebas en entorno local

En esta fase, abandonaremos el entorno online (ExpCov) y realizaremos una migración del código para trabajar en otro tipo de entorno. Se realizará la migración del algoritmo de C a Python, implementando exactamente la misma lógica en un archivo llamado triangulo.py. A continuación, se implementará un entorno de pruebas automatizadas integradas en Visual Studio Code.

Se han de realizar las siguientes acciones previas a documentar esta parte de la práctica:

- a. Se deberá configurar Visual Studio Code instalando la extensión "Coverage Gutters" (analizador de cobertura de condiciones) y las librerías de Python

necesarias (**pytest** y **pytest-cov**). Una vez configurado, se traducirá el código fuente original a Python, asegurando que la lógica de determinación del tipo de triángulo se mantiene intacta.

- b. Se creará un script de pruebas llamado `test_triangulo.py`, que tendrá que tener importada la librería **pytest**, y la función a comprobar desde el archivo `triangulo.py`. En este archivo, se deberán programar los mismos 5 casos de prueba utilizados en el apartado anterior (Parte 1), pero esta vez utilizando la sintaxis del framework `pytest`:

Ejemplo para definir un caso de prueba:

```
import pytest
from triangulo import checktriangle

def test_cas1_escaleno():
    assert checktriangle(6, 5, 10) == "Triangulo escaleno"
```

- c. Se ejecutarán las pruebas generando el reporte de cobertura XML necesario para que el IDE lo interprete. Para ello se deberán seguir los siguientes pasos:
 - En una terminal dentro del VisualStudio y estando el path en la misma carpeta que tus scripts de Python, ejecuta el paquete `pytest` para generar el XML de cobertura, con el siguiente comando:
`pytest --cov=. --cov-report=xml`
 - Una vez se genere el archivo XML en el mismo directorio (`coverage.xml`), se deberá activar la visualización de Coverage Gutters (función “Watch”) para observar directamente sobre el código fuente las líneas cubiertas (verde) y no cubiertas (rojo). “Watch” se encuentra en la barra azul de abajo del IDE de Visual Studio como un botón. El código mostrado en verde, estará cubierto, el rojo no estará cubierto con las pruebas actuales propuestas.

Documenta a continuación en tu práctica:

- a. Código migrado de C a Python explicando cómo has realizado la migración y si has tenido en cuenta algún factor en particular para traducir de un entorno (C) a otro (Python).
- b. Explica si has tenido que modificar algo en el código para que puedas aplicarle las pruebas desde el script automatizado.
- c. Captura de pantalla del IDE mostrando el código Python con las barras de color en el margen, evidenciando qué partes del código han sido ejecutadas por los 5 casos iniciales y la conclusión obtenida al hacer el apartado 3. ¿Qué diferencias aprecias con el resultado proporcionado por la herramienta ExpCov? ¿Por qué sucede esto?
- d. Analiza si es necesario añadir nuevos tests para alcanzar el 100% de cobertura.

Parte 3: Control de versiones

En esta parte de la práctica vamos a mantener el ciclo de vida del programa que hemos realizado mediante un control de versiones. Un control de versiones nos permitirá controlar la evolución del software, desde el código base, migración y el código en su versión definitiva, pasando por dejar constancia de los cambios realizados, por quién, y los problemas resueltos o por resolver, asociadas a las distintas versiones del programa. Esto nos permitirá cambiar la metodología de trabajo con metodologías de inventario, como ITIL, a otras metodologías más

modernas como son las DevOps, y a su vez cumplir con los requisitos de ISO/CMMi de “Control de Documentación” y “Control de Cambios”. Para alcanzar esto utilizaremos Git como control de versiones local y GitHub como repositorio remoto.

Se han de realizar las siguientes acciones previas a documentar esta parte de la práctica (no se proporciona el detalle de los comandos exactos puesto que requiere ser buscado por los alumnos):

Versión inicial del proyecto:

- 1) Crea un nuevo repositorio público en tu cuenta de GitHub llamado practica1-apellidos.
- 2) Inicializa un repositorio Git local en tu carpeta de trabajo.
- 3) Vincula tu repositorio local con el remoto que acabas de crear.
- 4) Sube únicamente el código original en C (triangulo.c) a la rama principal (main). Este commit representará la versión inicial del proyecto.

Registro de la migración del proyecto:

- 5) Sin crear ramas nuevas todavía, añade los archivos de la migración a Python (triangulo.py y test_triangulo.py).
- 6) Realiza un commit y súbelo a la nube. Esto simula que la migración ha sido aceptada en la línea base del proyecto.

Corrección de errores de proyecto:

- 7) Se ha detectado un bug en la lógica de los triángulos isósceles y una falta de cobertura.
- 8) Crea y cambia a una nueva rama específica para corregir este fallo (investiga convenciones de nombres para ramas de corrección, ej: fix/...).
- 9) En esa rama, corrige el código Python y añade los tests necesarios para el 100% de cobertura.
- 10) Sube tu nueva rama al repositorio remoto.

Integración del proyecto:

- 11) Desde la interfaz web de GitHub, abre una solicitud de integración (Pull Request) comparando tu rama de corrección contra la rama main.
- 12) Revisa visualmente las diferencias (diffs) para asegurar que solo cambias lo necesario.
- 13) Realiza la fusión (Merge) de la rama y cierra el ticket.

Documenta a continuación en tu práctica:

- a. En la memoria, poned un enlace a vuestro repositorio público de GitHub junto con los comandos y los pasos realizados para completar esta parte de la práctica.
- b. A continuación pon un “Network Graph” de GitHub, que encontrarás bajo el menú de Insights -> Network, y explica lo que ves en el mismo. Se valorará si se exportan o visualiza algún otro dato de GitHub relacionando la información con los distintos principios vistos en teoría.

Assignment (English)

Introduction

In this practice, we will learn about code coverage for testing and version control of our application. We will begin by analyzing code coverage in a legacy environment using C and

web-based analysis tools, to subsequently perform a migration to a modern ecosystem with Python and Visual Studio Code. In this environment, we will implement automated tests to visualize potential code issues in real-time. Finally, we will close the lifecycle by managing software evolution using Git and GitHub, simulating a collaborative workflow that ensures the traceability and integrity of every introduced change.

Tasks to perform

Part 1: Code coverage in a legacy environment

We will take the C code that implements a program to determine, given the lengths of its three sides, the type of triangle formed according to the lengths of its sides (scalene, equilateral or isosceles), also indicating if the data given really do not form a triangle. With this code, the following will be done:

The code will be tested using the [ExpCov](#) test coverage measurement website with the following input values:

- Case 1: 6, 5, 10
- Case 3: 3, 3, 4
- Case 2: 6, 6, 6
- Case 5: 4, 3, 0
- Case 4: 8, 2, 4

1) Verify and document the detailed coverage achieved with this set of cases, showing the tool's reports and indicators, as well as pointing out exactly which code options were not executed. Cases should be specified by indicating the input data, the expected output, and the actual output.

2) The minimum number of input cases necessary to attempt to achieve 100% coverage of decisions and conditions will be added. The additional cases shall be specified, justifying their choice based on a reasoned design. In addition, as in the previous phase, the coverage achieved and the indicators for the total set of cases (the previous ones plus the new ones added) shall be documented with details from the tool to demonstrate that complete coverage has been achieved.

3) A final testing phase will be carried out, in which additional test cases must be added to the previous minimums for total coverage and which are likely to detect problems in the code, **using only black-box logic of boundary values and special cases**. The logic for proposing the cases and their execution documentation (input used, expected correct output and actual output obtained) must be documented without reporting white-box coverage with ExpCov.

Code:

```
#include <math.h>
#include <stdio.h>

int
checktriangle (int a, int b, int c)
{
    int A, B, C;
    if ((c < a + b) && (b < a + c) && (a < c + b))
    {
        if (a == b && a == c)
```

```

        {printf ("Equilateral triangle \n");}
        else if (a == b || b == c)
            {printf ("Isosceles triangle \n");}
        else
            {printf ("Equal-sided triangle \n");}
    }
    else
        {printf ("It is not a triangle. \n");}
return 0;
}

int
main ()
{
    int a, b, c, i, n;
    printf ("Number of test cases: \n");
    scanf ("%d", &n);
    printf ("%d", n);
    printf ("\n");

    for (i = 1; i < n + 1; i++)
        {printf ("Mark the length values of the sides of the triangle (one per line): \n");
        scanf ("%d", &a); printf ("%d,", a);
        scanf ("%d", &b); printf ("%d,", b);
        scanf ("%d", &c); printf ("%d: \n", c);
        checktriangle (a, b, c);};
return 0;
}

```

Part 2: Migration and test coverage in a local environment

In this phase, we will leave behind the previous environment (ExpCov) and perform a code migration to work in a different type of environment. The algorithm will be migrated from C to Python, implementing the same exact logic in a file named `triangulo.py`. Next, an automated testing environment integrated into Visual Studio Code will be implemented.

The following actions must be performed prior to documenting this part of the practice:

- 1) Configure Visual Studio Code by installing the "Coverage Gutters" extension (condition coverage analyzer) and the necessary Python libraries (**pytest** and **pytest-cov**). Once configured, translate the original source code to Python, ensuring that the triangle type determination logic remains intact.
- 2) Create a test script named `test_triangulo.py`, which must import the `pytest` library and the function to be tested from the `triangulo.py` file. In this file, you must program the same 5 test cases used in the previous section (Part 1), but this time using the syntax of the `pytest` framework:

Example for defining a test case:

```

import pytest
from triangulo import checktriangle

def test_cas1_escaleno():
    assert checktriangle(6, 5, 10) == "Equal-sided triangle"

```

- 3) Execute the tests generating the XML coverage report necessary for the IDE to interpret it. To do this, follow these steps:
 - In a terminal inside Visual Studio, with the path set to the same folder as your Python scripts, execute the `pytest` package to generate the coverage XML using the following command: **`pytest --cov=. --cov-report=xml`**

- Once the XML file is generated in the same directory (coverage.xml), you must activate the Coverage Gutters visualization ("Watch" function) to directly observe the covered (green) and uncovered (red) lines on the source code. "Watch" is found on the blue bar at the bottom of the Visual Studio IDE as a button. Code shown in green is covered; red is not covered by the currently proposed tests.

Document the following in your practice report:

- a. Migrated code from C to Python, explaining how you performed the migration and if you took into account any particular factor when translating from one environment (C) to another (Python).
- b. Explain if you had to modify anything in the code to be able to apply the tests from the automated script.
- c. Screenshot of the IDE showing the Python code with the colored bars in the margin, evidencing which parts of the code were executed by the initial 5 cases, and the conclusion obtained from completing section 3. What differences do you appreciate compared to the result provided by the ExpCov tool? Why does this happen?
- d. Analyze if it is necessary to add new tests to reach 100% coverage.

Part 3: Version control

In this part of the practice, we will maintain the lifecycle of the program we have created using version control. Version control allows us to manage the evolution of the software, from the base code to the migration and the final definitive version. It allows us to keep a record of the changes made, by whom, and the issues resolved or pending, associated with the different versions of the program. This enables us to shift from inventory-based work methodologies, such as ITIL, to more modern methodologies like DevOps, while complying with ISO/CMMi requirements for "Documentation Control" and "Change Control". To achieve this, we will use Git for local version control and GitHub as the remote repository.

The following actions must be performed prior to documenting this part of the practice (exact command details are not provided as students are required to research them):

Initial project version:

1. Create a new public repository on your GitHub account named practica1-surnames.
2. Initialize a local Git repository in your working folder.
3. Link your local repository with the remote one you just created.
4. Upload only the original C code (triangulo.c) to the main branch (main). This commit will represent the initial version of the project.

Project migration record:

5. Without creating new branches yet, add the Python migration files (triangulo.py and test_triangulo.py).
6. Commit the changes and push them to the cloud. This simulates that the migration has been accepted into the project baseline.

Project bug fixing:

7. A bug has been detected in the isosceles triangle logic, along with a lack of coverage.
8. Create and switch to a new branch specifically to fix this failure (research naming conventions for fix branches, e.g., fix/...).
9. In that branch, fix the Python code and add the necessary tests to reach 100% coverage.
10. Push your new branch to the remote repository.

Project integration:

11. From the GitHub web interface, open a Pull Request comparing your correction branch against the main branch.
12. Visually review the differences (diffs) to ensure you are only changing what is necessary.
13. Perform the Merge of the branch and close the ticket.

Document the following in your practice report:

- a. In the report, include a link to your public GitHub repository along with the commands and steps taken to complete this part of the practice.
- b. Next, include a GitHub "Network Graph," which you will find under the *Insights* -> *Network* menu, and explain what you see in it. It will be positively valued if other GitHub data is exported or visualized, relating the information to the different principles seen in theory.